

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

My Screenshots –

Task_1:

1. Created a folder for Week4 skill

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~
ashborn@DESKTOP-I4G5Q4N:~$ mkdir Week4_s
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/Week4_s
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ pwd
/home/ashborn/Week4_s
```

2. Created a C-program file and compiled it.

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ nano task1.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ gcc task1.c -o task1
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls -l
total 36
-rwxr-xr-x 1 ashborn ashborn 16520 Aug 22 09:59 task1
-rw-r--r-- 1 ashborn ashborn  2328 Aug 22 09:59 task1.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$
```

Source Code –

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        /* Display shell prompt */
        printf("myShell> ");
        fflush(stdout);
        /* Read input */
        ssize_t bytes_read = read(STDIN_FILENO, input, sizeof(input) - 1);
        if (bytes_read <= 0)
        {
            printf("\nExiting shell...\n");
            break;
        }
        input[bytes_read] = '\0';

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';
        /* Check empty command */
        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }
        /* Tokenization */
        char *args[MAX_ARGS];
        int count = 0;
        char *token = strtok(input, " \t");
```

```

while (token != NULL && count < MAX_ARGS - 1)
{
    args[count] = token;
    count++;
    token = strtok(NULL, " \\t");
}
/* End argument list with NULL */
args[count] = NULL;
/* Validate tokens */
if (count == 0)
{
    printf("Invalid command.\n");
    continue;
}
/* Display parsed tokens */
printf("--- Parsed Tokens ---\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}
/* Create child process */
pid_t pid = fork();
if (pid < 0)
{
    perror("fork");
    continue;
}
/* Child process */
if (pid == 0)
{
    printf("Child PID: %d\n", getpid());
    /* Execute command */
    execvp(args[0], args);
    /* Only reached if execvp fails */
    perror("execvp");
    exit(EXIT_FAILURE);
}
/* Parent process */
else

```

```

    {
        printf("Parent PID: %d\n", getpid());
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }
}
return 0;
}

```

3. Executed the program.

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ./task1
myShell> echo Hello
--- Parsed Tokens ---
Argument 1: echo
Argument 2: Hello
Parent PID: 81
Child PID: 82
Hello
Child exited with status: 0
myShell> ls
--- Parsed Tokens ---
Argument 1: ls
Parent PID: 81
Child PID: 83
task1 task1.c
Child exited with status: 0
myShell> echo Operating Systems
--- Parsed Tokens ---
Argument 1: echo
Argument 2: Operating
Argument 3: Systems
Parent PID: 81
Child PID: 84
Operating Systems
Child exited with status: 0
myShell> █

```

Summary –

We split the user input into individual tokens by identifying spaces/whitespace and validated the command. We then used `fork()`, `execvp()`, and `waitpid()` to create a child process, execute the command, and make the parent wait for completion.

TASK_2:

1. Created a C-program and compiled it

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ nano task2.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ gcc task2.c -o task2
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls
task1 task1.c task2 task2.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$
```

Source code –

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    while (1)
    {
        printf("myShell> ");
        fflush(stdout);
        ssize_t bytes_read = read(STDIN_FILENO, input, sizeof(input) - 1);
        if (bytes_read <= 0)
        {
            printf("\nExiting shell...\n");
            break;
        }
        input[bytes_read] = '\0';
```

```

input[strespn(input, "\n")] = '\0';
if (strlen(input) == 0)
{
    printf("Empty command.\n");
    continue;
}
/* Check for unmatched single quotes */
int quote_count = 0;
for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
        quote_count++;
}
if (quote_count % 2 != 0)
{
    printf("Syntax Error: Unmatched single quote.\n");
    continue;
}
char *args[MAX_ARGS];
int count = 0;
char *p = input;
while (*p != '\0' && count < MAX_ARGS - 1)
{
    /* Skip whitespace */
    while (*p == ' ' || *p == '\t')
        p++;
    if (*p == '\0')
        break;
    /* Single quoted argument */
    if (*p == '"')
    {
        p++;
        args[count++] = p;
        while (*p != '"' && *p != '\0')
            p++;
        if (*p == '"')
        {
            *p = '\0';
            p++;
        }
    }
}

```

```

    }
    else
    {
        /* Normal argument */
        args[count++] = p;

        while (*p != ' ' && *p != '\t' && *p != '\0')
            p++;
        if (*p != '\0')
        {
            *p = '\0';
            p++;
        }
    }
}
args[count] = NULL;

printf("--- Parsed Arguments ---\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}
/* Create child process */
pid_t pid = fork();
if (pid < 0)
{
    perror("fork");
    continue;
}
if (pid == 0)
{
    printf("Child PID: %d\n", getpid());
    execvp(args[0], args);
    perror("execvp");
    exit(EXIT_FAILURE);
}
else
{
    int status;
    waitpid(pid, &status, 0);
}

```

```

        printf("Command execution completed.\n");
    }
}
return 0;
}

```

2. Executed the task

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ./task2
myShell> echo Hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child PID: 95
Hello
Command execution completed.
myShell> echo 'Hello World'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child PID: 96
Hello World
Command execution completed.
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
myShell>

```

Summary –

We implemented single-quote handling so text inside ' ' is preserved as a single argument, including spaces. We also detected unmatched single quotes and displayed a syntax error.

TASK_3:

1. Created a C-program and compiled it


```
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ nano task3.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ gcc task3.c -o task3
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls
task1 task1.c task2 task2.c task3 task3.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$
```

Source code –

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);
        ssize_t bytes_read = read(STDIN_FILENO, input, sizeof(input) - 1);
        if (bytes_read <= 0)
        {
            printf("\nExiting shell...\n");
            break;
        }
        input[bytes_read] = '\0';
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        /* Check for unmatched double quotes */
```

```

int quote_count = 0;
for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
        quote_count++;
}
if (quote_count % 2 != 0)
{
    printf("Syntax Error: Unmatched double quote.\n");
    continue;
}
char *args[MAX_ARGS];
int count = 0;
char *p = input;
while (*p != '\0' && count < MAX_ARGS - 1)
{
    /* Skip whitespace */
    while (*p == ' ' || *p == '\t')
        p++;
    if (*p == '\0')
        break;
    /* Double quoted argument */
    if (*p == '"')
    {
        p++;

        args[count++] = p;

        while (*p != '"' && *p != '\0')
            p++;
        if (*p == '"')
        {
            *p = '\0';
            p++;
        }
    }
    else
    {
        /* Normal argument */
        args[count++] = p;
    }
}

```

```

        while (*p != ' ' && *p != '\t' && *p != '\0')
            p++;
        if (*p != '\0')
        {
            *p = '\0';
            p++;
        }
    }
}
args[count] = NULL;
/*
 * Expand $USER inside parsed arguments.
 */
for (int i = 0; i < count; i++)
{
    char *user_pos = strstr(args[i], "$USER");
    if (user_pos != NULL)
    {
        char expanded[MAX_INPUT];
        char *username = getenv("USER");
        if (username != NULL)
        {
            int prefix_length = user_pos - args[i];
            snprintf(
                expanded,
                sizeof(expanded),
                "%.*s%s%s",
                prefix_length,
                args[i],
                username,
                user_pos + 5
            );
            strcpy(args[i], expanded);
        }
    }
}
printf("--- Parsed Arguments ---\n");
for (int i = 0; i < count; i++)
{

```

```

        printf("Argument %d: %s\n", i + 1, args[i]);
    }
    /* Create child process */
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        continue;
    }
    /* Child process */
    if (pid == 0)
    {
        printf("Child Process PID = %d\n", getpid());
        execvp(args[0], args);
        perror("execvp");
        exit(EXIT_FAILURE);
    }
    /* Parent process */
    else
    {
        int status;

        waitpid(pid, &status, 0);

        printf("Child process completed.\n");
    }
}
return 0;
}

```

2. Executed the task.

Summary –

We implemented double quotes so spaces inside " " are preserved while allowing variable expansion such as \$USER. We also added detection for unmatched double quotes.

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ./task3
myShell> echo "Hello World"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child Process PID = 105
Hello World
Child process completed.
myShell> echo "Hello $USER"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello ashborn
Child Process PID = 106
Hello ashborn
Child process completed.
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
myShell> █

```

TASK_4:

1. Created a C-program and compiled it

```

ashborn@DESKTOP-I4G5Q4N:~$ cd ~/Week4_s
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls
task1 task1.c task2 task2.c task3 task3.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ nano task4.c

```

Source code –

```

#include <stdio.h>
#include <string.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];
    printf("myShell$ ");
    fflush(stdout);
    if (fgets(input, sizeof(input), stdin) == NULL)

```

```

{
    printf("No input received.\n");
    return 1;
}
input[strcspn(input, "\n")] = '\0';
if (strlen(input) == 0)
{
    printf("Empty command.\n");
    return 0;
}
char *args[MAX_ARGS];
int count = 0;
char *p = input;
while (*p != '\0' && count < MAX_ARGS - 1)
{
    /* Skip whitespace between arguments */
    while (*p == ' ' || *p == '\t')
        p++;
    if (*p == '\0')
        break;
    /* Beginning of a new argument */
    args[count++] = p;

    char *write = p;

    while (*p != '\0')
    {
        /* Handle escape sequence */
        if (*p == "\\")
        {
            p++;
            if (*p != '\0')
            {
                *write = *p;
                write++;
                p++;
            }
        }
        /* Normal whitespace ends the argument */
        else if (*p == ' ' || *p == '\t')

```

```

    {
        /*
         * Move p past the whitespace BEFORE
         * replacing it with '\0'.
         */
        p++;
        break;
    }
    else
    {
        *write = *p;
        write++;
        p++;
    }
}
/* Terminate the argument */
*write = '\0';
}
args[count] = NULL;
printf("--- Parsed Arguments ---\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}
return 0;
}

```

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ gcc task4.c -o task4
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls
task1 task1.c task2 task2.c task3 task3.c task4 task4.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ █

```

2. Executed the program

Summary –

We implemented escape-sequence handling using `\` so special characters and spaces can be treated as literal characters. For example, `Hello\ World` is parsed as one argument: `Hello World`.

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ printf '%s\n' 'echo Hello\ World' | ./task4
myShell$ --- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ printf '%s\n' 'file\ name.txt' | ./task4
myShell$ --- Parsed Arguments ---
Argument 1: file name.txt
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ printf '%s\n' 'Hello\ World\ Again' | ./task4
myShell$ --- Parsed Arguments ---
Argument 1: Hello World Again
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ █

```

TASK_5:

1. Created a new C-program and compiled it.

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ nano task5.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ gcc task5.c -o task5
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ls
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c

```

Source code –

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);
        /* Read command using read() */
        ssize_t bytes_read = read(STDIN_FILENO, input, sizeof(input) - 1);
        if (bytes_read <= 0)
        {

```



```

        printf("\nShell terminated.\n");
        break;
    }
    input[bytes_read] = '\0';
    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';
    /* Handle empty command */
    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        continue;
    }
    /* Tokenize command and arguments */
    char *args[MAX_ARGS];
    int count = 0;
    char *token = strtok(input, " \t");
    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;

    /* Validate command */
    if (count == 0)
    {
        printf("Invalid command.\n");
        continue;
    }
    /* Create child process */
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }
    /* CHILD PROCESS */
    if (pid == 0)
    {

```

```

printf("Child Process PID: %d\n", getpid());

/*
 * Replace child process with requested program.
 * args contains command + all arguments.
 */
execvp(args[0], args);

/*
 * This is reached only when execvp() fails.
 */
perror("execvp");
exit(2);
}
/* PARENT PROCESS */
else
{
printf("Parent Process PID: %d\n", getpid());
printf("Created Child PID: %d\n", pid);

int status;
/* Wait for child to finish */
waitpid(pid, &status, 0);
/* Get child's exit status */
if (WIFEXITED(status))
{
printf("Child exited with status: %d\n",
WEXITSTATUS(status));
}
}
}
return 0;
}

```

2. Executed the program

```

ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ ./task5
myshell$ ls
Parent Process PID: 205
Child Process PID: 206
Created Child PID: 206
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
Child exited with status: 0
myshell$ ls -l
Parent Process PID: 205
Child Process PID: 207
Created Child PID: 207
total 128
-rwxr-xr-x 1 ashborn ashborn 16520 Aug 22 09:59 task1
-rw-r--r-- 1 ashborn ashborn 2328 Aug 22 09:59 task1.c
-rwxr-xr-x 1 ashborn ashborn 16480 Aug 22 10:10 task2
-rw-r--r-- 1 ashborn ashborn 2699 Aug 22 10:10 task2.c
-rwxr-xr-x 1 ashborn ashborn 16632 Aug 22 10:16 task3
-rw-r--r-- 1 ashborn ashborn 3593 Aug 22 10:16 task3.c
-rwxr-xr-x 1 ashborn ashborn 16272 Aug 29 08:26 task4
-rw-r--r-- 1 ashborn ashborn 1800 Aug 29 08:26 task4.c
-rwxr-xr-x 1 ashborn ashborn 16520 Aug 29 08:35 task5
-rw-r--r-- 1 ashborn ashborn 2331 Aug 29 08:34 task5.c
Child exited with status: 0
myshell$ echo Hello OS
Parent Process PID: 205
Created Child PID: 208
Child Process PID: 208
Hello OS
Child exited with status: 0
myshell$ ls @l
Parent Process PID: 205
Child Process PID: 209
Created Child PID: 209
ls: cannot access '@l': No such file or directory
Child exited with status: 2
myshell$ ^C
ashborn@DESKTOP-I4G5Q4N:~/Week4_s$ █

```

Summary –

We created a child process using `fork()` and used `execvp()` to execute commands with their arguments, while the parent uses `waitpid()` to wait for the child. We also handled execution errors and displayed the child's exit status, such as 0 for success and 2 for the tested error case.